Тема 11: Процедурно програмиране – основни конструкции

Анотация:

- Принципи на структурното програмиране.
- Управление на изчислителния процес.
- Основни управляващи конструкции условни оператори, оператори за цикъл.
- Променливи
    - локални променливи
    - глобални променливи;
    - инициализация на променлива
    - оператор за присвояване.
- Функции и процедури.
    - Параметри и видове параметри.
    - Предаване на параметри по име и по стойност.
    - Типове и проверка за съответствие на тип

Принципи на структурното програмиране

В програмирането съществуват различни парадигми за изграждане на програми. Една от най-ранните и най-широко използвани е процедурната парадигма. Тя служи като основа за разбирането на съвременни подходи и е неизменна част от изучаването на алгоритми и структури от данни.

Императивно и декларативно програмиране

Програмирането може да бъде разделено на две основни парадигми:

- Императивно програмиране
    - описва как се изпълнява дадена задача, чрез поредица от инструкции, които променят състоянието на програмата (например чрез променливи и команди за контрол на потока).
    - Тази парадигма включва процедурното и структурното програмиране.
- Декларативно програмиране
    - Описва какво трябва да се постигне, без да се посочват конкретните стъпки за изпълнение.
    - Примери за декларативни езици са SQL за бази данни и функционалното програмиране.

Процедурното програмиране (определение)

Процедурното програмиране е императивна парадигма, при която програмата се изгражда от последователно изпълнявани инструкции, организирани във функции (наричани още процедури). Основният подход е линеен (отгоре надолу), като данните и функциите се третират като отделни компоненти. Всяка функция изпълнява конкретна подзадача и може да бъде извиквана многократно.

Принципи на структурното програмиране

Структурното програмиране е разширение на процедурното, което поставя акцент върху ясната, четлива и логично организирана структура на програмния код. Основна цел е да се избягва хаотичното използване на безусловни преходи (например goto), които затрудняват разбирането и поддръжката на програмите. Вместо това се използват конструкции като цикли, условия и функции, които оформят програмата в добре дефинирани блокове.

Основни характеристики на процедурното програмиране

- Функции (процедури)

```
<тип> <име_на_функция>(<параметри>) {
    <тяло_на_функцията>
}
```

    - инструкции, които се характеризират със сигнатура и тяло
    - Те могат да бъдат:
        - дефинирани от потребителя;
        - предоставени от стандартни или външни библиотеки.

- Променливи

```
<тип> <име_на_променлива> = <стойност>;
```

    - Локални променливи
        - променливи, които са декларирани в основната структура на функцията и са ограничени до локалния обхват.
        - Може да се използват само във функцията, в който са дефинирани;
    - Глобални променливи
        - глобалните променливи се декларират извън функциите и са достъпни за всички функции в програмата.

- Модулност
    - Програмата се разделя на отделни логически части (модули), които комуникират чрез функции.
        - Различни модули изпълняват различни подзадачи, които заедно изграждат решението на една обща, по-сложна задача. Това подобрява четимостта и улеснява поддръжката на кода.
        - След това всяка група системи ще има свои собствени задачи, завършени една след друга, докато всички задачи не бъдат изпълнени.

```
<тип> функция1(...) { ... }
<тип> функция2(...) { ... }
int main() {
    // извиквания към функции
}
```

- Предаване на параметри
    - механизъм, използван за предаване на параметри към функции, подпрограми или процедури.
    - Предаването на параметър може да се извърши чрез:
        - предаване по стойност

```
<тип> <име_на_функция>(<тип> параметър) { ... }
```

        - предаване чрез указател

```
<тип> <име_на_функция>(<тип>* параметър) { ... }
```

        - предаване чрез референция

```
<тип> <име_на_функция>(<тип>& параметър) { ... }
```

Предимства

- Ясен и последователен поток на управление;
- Повторна употреба на код чрез функции;
    - Кодът може да се използва повторно в различни части на програмата, без да е необходимо да се копира;
- По-добро управление на паметта.
    - Редуциране на използваната памет;
- Преносимост на кода между различни среди.
    - Изходният код е преносим, следователно може да се използва и за насочване към различни процесори.

Недостатъци

- Ограничена структурна гъвкавост при големи проекти;
    - По-труден за писане код, особено ако обема нараства, тъй като липсва йерархия;
- Трудности при моделиране на реалния свят;
    - Трудно се свързва с обекти от реалния свят;
- Данните са изложени на цялата програма, което я прави не толкова защитена

Управление на изчислителния процес

В процедурното програмиране програмната структура се състои от ясно дефинирани елементи, които изграждат логиката и потока на изпълнение на програмата. Тези елементи представляват основните градивни блокове, чрез които се съставят алгоритми, реализират се решения и се създават пълноценни програми.

Инструкция (statement)

Инструкцията е най-елементарната единица за изпълнение в една програма. Това е команда, която казва на компютъра да извърши определено действие. В езика C++ всяка инструкция завършва с точка и запетая `;`.

Някои основни типове инструкции:

- Присвояване: `a = 5;`
- Извикване на функция: `print();`
- Въвеждане/извеждане: `cin >> x; cout << x;`
- Условна инструкция: `if (x > 0) cout << "положително";`

Израз (expression)

Изразът представлява комбинация от:

- постоянни стойности (константи) - напр. `3.14`, `'A'`, `true`;
- променливи – напр. `a`, `n`, `sum`;
- оператори – напр. `+`, `-`, `*`, `=`, `==` и др.

Изразът винаги има стойност, която се изчислява в момента на изпълнение. Примери за изрази:

```
a + b;
x > y;
count++;
n % 2 == 0;
```

Изразът може да бъде част от инструкция, напр.:

```
int z = x + y * 2;
```

Блок от инструкции (compound statement)

Блокът от инструкции (или съставна инструкция) позволява да се групират няколко отделни инструкции и да се третират като една логическа единица. Това се постига чрез заграждане на инструкциите с фигурни скоби `{}`.

Блоковете се използват най-често:

- в тялото на условни конструкции (if, else, switch);
- в цикли (for, while);
- във функции – тялото на функцията е блок.

```
{
    <инструкция1>;
    <инструкция2>;
    ...
}
```

Коментари

Коментарите са части от кода, предназначени за обяснение, пояснение или временно деактивиране на код. Те не се изпълняват от компилатора. В езика C++ се използват два вида коментари:

- Едноредови

```
// Това е коментар
```

- Многоредови

```
/* текст */
```

Значение:

- Подобряват четимостта на кода;
- Улесняват поддръжката;
- Помагат при работа в екип;
- Полезни при временно изключване на части от кода.

Условни конструкции

Условните конструкции позволяват изпълнението на различни блокове от код в зависимост от стойността на дадено условие (булев израз). Използват се за разклоняване на логиката.

- if конструкция
    - Изпълнява блок от код само ако условието е вярно.

```
if (условие) {
    // инструкции при вярно условие
}

// Пример
if (x > 0) {
    std::cout << "Положително число";
}
```

- if-else конструкция
    - Изпълнява един блок, ако условието е вярно, и друг – ако е невярно.

```
if (условие) {
    // инструкции при вярно условие
} else {
    // инструкции при невярно условие
}

// Пример
if (x % 2 == 0) {
    std:: cout << "Четно число";
} else {
    std::cout << "Нечетно число";
}
```

- if - else if - else
    - Използва се, когато има повече от две възможности (разклонения).

```
if (условие1) {
    // код при условие1
} else if (условие2) {
    // код при условие2
} else {
    // код, ако никое от горните не е вярно
}

// Пример
if (x < 0) {
    std::cout << "Отрицателно";
} else if (x == 0) {
    std::cout << "Нула";
} else {
    std::cout << "Положително";
}
```

- switch конструкция
    - Използва се, когато трябва да се избере между множество цели стойности на дадена променлива (обикновено цяло число или символ).

```
switch (израз) {
case стойност1:
    // инструкции
    break;
case стойност2:
    // инструкции
    break;
...
default:
    // инструкции при липса на съвпадение
}

// Пример
int day = 3;
switch (day) {
    case 1: cout << "Понеделник"; break;
    case 2: cout << "Вторник"; break;
    case 3: cout << "Сряда"; break;
    default: cout << "Невалиден ден";
}
```

- Тернарен оператор (?:)
    - Тернарният оператор е съкратена форма на условна конструкция if-else, използвана за присвояване на стойност или избор между две алтернативи в едно изречение (израз).
    - Нарича се тернарен, защото работи с три операнда.

```
условие ? стойност_ако_е_вярно : стойност_ако_е_невярно;

// Пример
int a = 5, b = 10;
int max = (a > b) ? a : b;
```

Цикли (повторения)

Циклите се използват за многократно изпълнение на един и същи блок от код, докато дадено условие е вярно.

- for цикъл
    - Подходящ, когато броят на итерациите е известен предварително.

```
for (инициализация; условие; стъпка) {
    // инструкции
}

// Пример
for (int i = 0; i < 5; i++) {
    std::cout << i << " ";
}
```

- while цикъл
    - Използва се, когато броят на повторенията не е известен предварително, а зависи от дадено условие.

```
while (условие) {
    // инструкции
}

// Пример
int x = 1;
while (x <= 5) {
    cout << x << " ";
    x++;
}
```

- do...while цикъл
    - Изпълнява кода поне веднъж, дори ако условието не е вярно в началото.
    - Проверява условието след първото изпълнение.

```
do {
    // инструкции
} while (условие);

// Пример
int x = 0;
do {
    cout << x << " ";
    x++;
} while (x < 3);
```

Оператори за контрол на потока

Тези оператори управляват потока на изпълнение вътре в цикли или функции.

- break
    - Прекъсва изпълнението на най-близкия цикъл или switch конструкция.

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    cout << i << " ";
}
```

- continue
    - Прекъсва текущата итерация и преминава към следващата.

```
for (int i = 0; i < 5; i++) {
    if (i == 2) continue;
```

```
        cout << i << " ";
    }
```

- return
    - Прекъсва изпълнението на функция и връща стойност (или просто излиза от void функция).

```
    int max(int a, int b) {
        if (a > b) return a;
        else return b;
    }
```

Променливи и типове данни

Програмите работят с данни, които се съхраняват в променливи. Всяка променлива има име, тип и стойност, като типът определя размера и вида на допустимите стойности, както и операциите, които могат да се извършват с тях.

```
<тип> <име> = <стойност>;
```

Скаларни типове

Скаларни типове в програмирането са прости типове данни, които съдържат само една стойност наведнъж. Основните скаларни типове в C++ са:

- int - цяло число, поне 16 бита, със знак (положителни и отрицателни стойности).
- long - цяло число, поне 32 бита, размерът зависи от платформата (обикновено 32 или 64 бита).
- long long - цяло число с гарантиран 64-битов размер.
- unsigned - цели числа без знак (само положителни и нула).
- float - дробно число, 32 битово, с плаваща запетая, с около 7 десетични знака точност.
- double - дробно число, 64 битово, с плаваща запетая, с около 15 десетични знака точност (двойна точност).
- char – единични символи (буква, цифра или специален знак).
- bool – логически тип (true/false).

Деклариране и инициализация:

```
int a = 5;
float pi = 3.14;
char ch = 'A';
bool flag = true;

int b = pi; //3, преобразуване със загуба на информация
```

Обхват и време на живот на променливите

Променливите съществуват в определен обхват (scope) и имат определен времеви живот (lifetime):

- Локални променливи - дефинирани вътре в функции, достъпни само там;

```
void func() {
    int local = 5; // валидна само вътре в func
}
```

- Глобални променливи - дефинирани извън функции, достъпни във всички функции;

```
int global = 10;

void print() {
    std::cout << global; // достъпна отвсякъде
}
```

- Автоматични променливи (по подразбиране локални);
    - локални променливи, създават се при влизане в блок и се унищожават при излизане.
    - Ключовата дума auto също може да се използва за автоматично определяне на типа:

```
auto x = 3.14; // x е от тип double
```

- Статични променливи - запазват стойността си между извиквания на функцията.

```
void counter() {
    static int count = 0;
    count++;
    std::cout << count << std::endl;
}
```

Оператори

Операторите са символи или комбинации от символи, които извършват операции върху една или повече стойности (операнди). В C++ операторите се делят на няколко групи според предназначението и броя на операндите. Всеки оператор се характеризира с:

- позиция на оператора спрямо аргументите му;
- приоритет;
- асоциативност.

Оператор за присвояване

Операторът за присвояване в програмирането е специален символ или команда, която служи за присвояване на стойност на променлива. Той осъществява копиране на данни от израз, разположен отдясно на оператора, към променлива, посочена отляво. В повечето програмни езици операторът за присвояване се означава с един знак =.

```
variable = expression;
```

- variable
    - идентификаторът на променливата, на която ще бъде присвоена стойност.
- expression
    - израз, чиято стойност ще се изчисли и присвои.

Функционалност: При изпълнение на оператор за присвояване се извършват следните стъпки:

- Изчислява се стойността на израза вдясно от оператора. Тази стойност може да бъде константа, резултат от друга променлива, функция или сложен израз.
- Получената стойност се копира в паметния адрес, свързан с променливата вляво от оператора.
- В резултат променливата приема новата стойност, заменяйки евентуално предишната си стойност.

```
int x;       // Деклариране на цяло число x
x = 10;      // Присвояване на стойност 10 на променливата x
int y = x;   // Присвояване на текущата стойност на x на променливата y
y = y + 5;   // Присвояване на резултата от израза y+5 на променливата y
```

В този пример:

- Първият оператор x = 10; задава стойността 10 на променливата x.
- След това y = x; копира стойността на x в y.
- Накрая y = y + 5; изчислява израза и присвоява резултата на y.

Стойността, която се присвоява, трябва да бъде съвместима с типа на променливата. Ако типовете не съвпадат, компилаторът може да издаде грешка или да извърши неявно преобразуване.

Видове оператори според броя на операндите (Допълнние, ако има време)

- Унарен - Работи с един операнд | Оператор | Описание | Пример | Забележка
  | |----------|----------------------------|------------|--| | `+` | Унарно плюс | `+x` | Почти неутрален | | `-` | Унарно минус | `-x` | Инвертира знака на стойността | | `++` | Инкремент (увеличение с 1) | `++x`, `x++` | Префиксен или постфиксен | | `--` | Декремент (намаление с 1) | `--x`, `x--` | Префиксен или постфиксен | | `!` | Логическо отрицание | `!flag` | Връща `true`, ако `flag` е `false` | | `~` | Побитово отрицание | `~x` | Инвертира всички битове |
  - Пояснение: `++x` увеличава стойността и връща новата стойност, докато `x++` връща старото число, преди да увеличи.
- Бинарен
  - Работи с два операнда
  - Аритметични оператори | Оператор | Описание | Пример | Резултат
    | |----------|----------------------|------------|-----------------------------| | `+` | Събиране | `a + b` | Сумата на `a` и `b` | | `-` | Изваждане | `a - b` | Разликата между `a` и `b` | | `*` | Умножение | `a * b` | Произведение на `a` и `b` | | `/` | Деление | `a / b` | Частно (цяло или реално) | | `%` | Остатък от деление | `a % b` | Остатък при целочислено деление |

- Логически оператори | Оператор | Описание | Пример | Резултат |
  |----------|----------|-------------------|--|
  | `&&` | И | `a > 0 && b < 5` | `true`, ако и двете условия са изпълнени |
  | `||` | ИЛИ | `x == 1 || y == 2` | `true`, ако поне едно е вярно |
- Оператори за сравнение | Оператор | Описание | Пример |
  |----------|--------------------|-----------|
  | `==` | Равно | `a == b` |
  | `!=` | Различно | `a != b` |
  | `<` | По-малко | `a < b` |
  | `>` | По-голямо | `a > b` |
  | `<=` | По-малко или равно | `a <= b` |
  | `>=` | По-голямо или равно | `a >= b` |
- Оператори за присвояване | Оператор | Описание | Пример | Еквивалент на
  |----------|----------------------------------|--------------|-----------------------|
  | `=` | Присвояване | `a = 5` | Задава стойност |
  | `+=` | Прибавяне и присвояване | `a += 3` | `a = a + 3` |
  | `-=` | Изваждане и присвояване | `a -= 2` | `a = a - 2` |
  | `*=` | Умножение и присвояване | `a *= 4` | `a = a * 4` |
  | `/=` | Деление и присвояване | `a /= 2` | `a = a / 2` |
  | `%=` | Остатък и присвояване | `a %= 3` | `a = a % 3` |
  | `&=` | Побитово И и присвояване | `a &= b` | `a = a & b` |
  | `\|=` | Побитово ИЛИ и присвояване | `a \|= b` | `a = a \| b` |
  | `^=` | Побитово XOR и присвояване | `a ^= b` | `a = a ^ b` |
  | `<<=` | Побитово изместване вляво и присв. | `a <<= 1` | `a = a << 1` |
  | `>>=` | Побитово изместване вдясно и присв. | `a >>= 1` | `a = a >> 1` |
- Побитови оператори | Оператор | Описание | Пример | Действие
  |----------|--------------------------------|-------------|---------------------------------------|
  | `&` | Побитово И (AND) | `a & b` | 1 само ако и двата бита са 1 |
  | `\|` | Побитово ИЛИ (OR) | `a \| b` | 1 ако поне един бит е 1 |
  | `^` | Побитово XOR (изключ. ИЛИ) | `a ^ b` | 1 ако битовете са различни |
  | `<<` | Побитово изместване наляво | `a << 2` | Измества битовете на `a` 2 позиции наляво |
  | `>>` | Побитово изместване надясно | `a >> 1` | Измества битовете на `a` 1 позиция надясно |

- Тернарен
  - Работи с три операнда
  - Пример: Тернарен оператор - `условие ? стойност1 : стойност2`

Функции и процедури

Функциите са основни програмни единици, които позволяват групиране на код за многократно използване. Когато даден код се използва на повече от едно място, той се оформя като функция, която може да бъде извиквана многократно. Това подобрява четимостта и поддръжката на програмата, като намалява излишъка от повтарящ се код.

В езика C++ терминът процедура не съществува като отделна конструкция – използва се функция с тип void, която не връща стойност. Такива функции често се наричат „процедури“ в учебен контекст.

Основни характеристики на функцията

- Сигнатура на функцията
    - Уникален идентификатор на функцията, който включва:
        - името на функцията;
        - броя и типовете на формалните параметри;
        - типа на връщаната стойност.
    - Служи за различаване на функции с еднакво име (претоварване на функции).
- Тип на връщаната стойност
    - Определя какъв тип данни функцията връща при завършване на изпълнението.
    - Ако не връща стойност, типът е void.
- Формални параметри (аргументи)
    - Входни данни, които функцията приема при извикване.
    - Предават се:
        - по стойност – копие на аргумента;
        - по референция – достъп до оригинала;
        - със стойности по подразбиране – използват се, ако не са подадени при извикване.
- Тяло на функцията
    - Състои се от последователност от инструкции, които се изпълняват при извикване.
    - Може да съдържа локални променливи, управляващи конструкции и изрази за връщане на стойност (return).

Структура на функция

```
[<модификатор>] <тип_на_връщане> <име>(<формални_параметри>) {
    // тяло на функцията
}
```

- Тип на функцията
    - описва типа на върнатата стойност, например int, float, void.
- Име
    - служи за идентификация и извикване.
- Формални параметри
    - списък с аргументи, които функцията приема. Могат да имат стойности по подразбиране, които се използват, ако при извикването не се подадат конкретни стойности.

Примери за функции:

```
// Дефиниране на функция без аргументи и без да връща стойност.
void print() {
    std::cout << "print text\n";
}

print(); // извикване на функция

// Функция с аргументи по подразбиране:
int sum(int x, int y, int z = 1, int w = 2) {
 return x + y + z + w;
}

// Функция, приемаща параметър по референция (псевдоним)
// Функцията работи с оригиналната променлива
void increment(int &x) {
    ++x;
}
```

Видове предаване на параметри

- По стойност (by value)
    - Предава се копие на аргумента. Промените във функцията не засягат оригинала.

```
  void foo(int x) { x = 10; } // оригиналната стойност не се променя
```

- По референция (by reference)
    - Предава се псевдоним на оригиналния аргумент. Промените влияят директно на извънфункционалната стойност.

```
  void foo(int& x) { x = 10; } // x се променя извън функцията
```

- По указател (pointer)
    - Предава се адрес на стойност. Изисква разименуване, но работи подобно на референция.

```
  void foo(int* x) { *x = 10; } // отново променя оригиналната стойност
```

Пояснение: Указателите са променливи, които съхраняват адреси на други променливи в паметта и позволяват индиректен достъп до тях. Референциите предоставят по-безопасен начин за достъп до променливи по адрес. Използват се при предаване на параметри и динамично заделяне на памет.

Типове и проверка за съответствие на тип

- При извикване на функция, типовете на аргументите трябва да съвпадат с типовете на параметрите.
- C++ позволява автоматично преобразуване на типове, но това може да доведе до нежелани резултати.
- Строгото съвпадение на типове се препоръчва при претоварване на функции или използване на шаблони.

```
void printInt(int x);
```

```
printInt(3.14); // компилаторът преобразува 3.14 до 3
```

Анонимна фунцкия/Ламбда функция

Функциите може и да са анонимни без име или наричани още ламбда функции. Обща дефиниция на ламбда функция:

```
[<прихващащ клас>] (<формални параметри>) {
 <тяло>
}
```

Прихващащият клас може да е [&] или [=].

- [&] означава, че променливите са прихванати по референция
- [=] означава, че променливите са прихванати по стойност.
- По подразбиране прихващащият клас е [=]. Пример: Сортиране на масив с анонимна функция функция

```
int arr[ ] = {3, 1, 2};
std::sort(begin(arr), end(arr), [&]( int a,  int b) { return a < b; });
```

Предимства на функциите

- Повторно използване на код;
- Подобрява четимостта и структурата;
- Улеснява поддръжката и тестването;
- Подпомага модулността и разделянето на задачите на логически части;